

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Test Structure & Naming Conventions Guide

Organise tests so they stay readable as your suite grows.

Introduction

A small suite is easy; a growing one becomes a mess without conventions. This guide gives you simple rules for structuring and naming tests so the suite stays clear and maintainable.

Folder & File Structure

- Group tests by feature/area (e.g. tests/auth, tests/checkout).
- One spec file per feature or user journey.
- Keep page objects, fixtures and data in their own folders.

Naming Conventions

Item	Convention	Example
Spec file	feature.spec.ts	login.spec.ts
describe block	Feature / area	describe('Login')
Test name	behaviour + expectation	'rejects an invalid password'
Page object	PascalCase + Page	LoginPage

Worked Example

`test.describe('Checkout', () => { test('applies a valid discount code', ...); test('rejects an expired discount code', ...); });` — anyone reading the report instantly understands what is covered.

Practical Exercise

Reorganise a small set of tests into feature folders and rename them to describe behaviour and expectation. Re-read the report — is it clearer?

Best Practices

- Name tests so a failure report reads like a sentence.
- Group with describe blocks.
- Keep one behaviour per test.

Common Mistakes

- Generic names (test1, works).
- Giant spec files covering everything.
- No grouping, so reports are hard to scan.

INSIDE STLC PRO TIP

Write test names so that when one fails, the report alone tells you what broke — "Checkout › rejects an expired discount code" needs no investigation to understand.